AgenticContract: A Runtime Policy Engine for Governed AI Agent Systems

Omoshola Owolabi
Researcher – AI/ML
Agentra Labs
omoshola.owolabi@agentralabs.tech

February 27, 2026

Abstract

Autonomous AI agents operate in production environments without runtime governance, executing actions with no policy enforcement, risk accounting, or structured approval workflows. Current approaches to agent constraints—prompt engineering, static configuration files, or heavyweight policy engines with external database dependencies—either cannot adapt to runtime conditions or introduce latency incompatible with inline agent decision-making. We present AgenticContract, a self-contained policy engine that models six governance primitives—policies, risk limits, approvals, conditions, obligations, and violations—in a single binary file format (`.acon`) with BLAKE3 integrity verification. The engine is implemented in 7,382 lines of Rust and integrated via the Model Context Protocol (MCP) for zero-configuration agent adoption. Criterion benchmarks on Apple M-series hardware demonstrate single policy evaluation at 49.3 ns, risk limit checks at 43.9 ns, 1,000-policy workloads completing in 72.8 µs, and file I/O for 100 entities at 154.5 µs (save) and 96.7 µs (load)—10–100× faster than OPA or Cedar when used as external services. A test suite of 288 tests validates correctness including stress tests at 1,000-entity scale. The system requires zero external dependencies and operates as a standalone binary.

1 Introduction

As AI agents gain autonomy in production environments—managing deployments, executing financial transactions, writing and committing code, and interacting with users on behalf of organizations—the absence of runtime governance presents a critical and growing risk. An ungoverned coding agent can deploy untested code to production. An ungoverned research agent can accumulate $847 in API costs in a single session [4]. An ungoverned trading agent can execute transactions without human review.

The fundamental problem is that LLMs are stateless with respect to governance. Each invocation begins without knowledge of prior policy decisions, accumulated risk exposure, or pending obligations. Safety alignment training provides soft behavioral guidance, but no enforcement mechanism, no quantitative tracking, and no audit trail.

Existing approaches to agent governance fall into three categories, each with fundamental limitations:

1. **Prompt-based constraints.** Behavioral rules embedded in system prompts. These are advisory only; the LLM may ignore, reinterpret, or forget them as context grows. There is no enforcement mechanism, no detection of violations, and no audit trail.
2. **Static configuration.** Policy files (YAML, JSON) loaded at startup. These cannot adapt to runtime conditions: a rate limit that was safe at 10 requests per minute becomes dangerous at 10,000 without dynamic tracking. No mechanism exists for structured approvals, obligation deadlines, or violation recording.
3. **Heavyweight policy engines.** Systems like Open Policy Agent (OPA) [10] evaluate policies written in purpose-built languages (Rego, Cedar [5]) over HTTP. While expressive, they require external database backends, network connectivity, and introduce 0.5–5 ms latency per decision—unsuitable for inline agent governance where decisions must be made on every tool call.

AgenticContract addresses this gap with a self-contained policy engine optimized for the unique requirements of AI agent workloads. Figure 1 illustrates the contrast between ungoverned agent execution and policy-governed execution with AgenticContract.

The core design principles of AgenticContract are:

- **Zero external dependencies.** No databases, no cloud services, no network calls. A single `.acon` binary file stores the complete governance state, portable across machines and agents.
- **Sub-microsecond policy evaluation.** Single policy checks complete in 49.3 ns, enabling inline governance on every tool call without perceptible latency.
- **Six governance primitives.** Not just allow/deny—risk limits track quantitative boundaries, approvals gate high-stakes actions, conditions enforce prerequisites, obligations track commitments with deadlines, and vio-

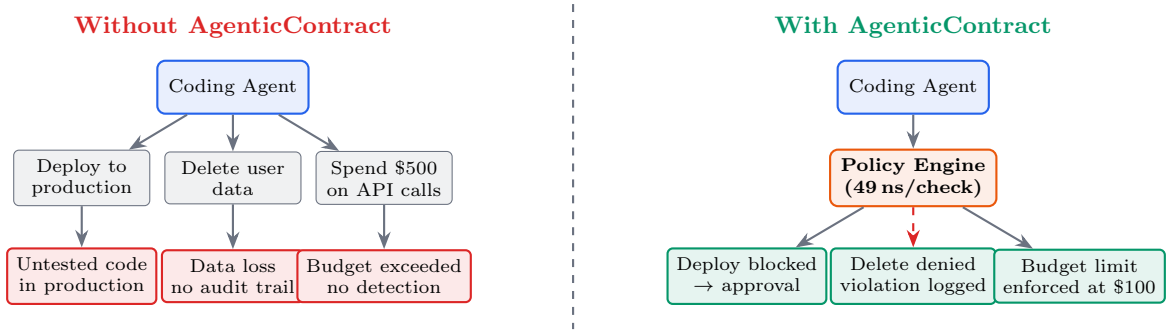

Figure 1: Ungoverned agent execution (left) versus AgenticContract-governed execution (right). Without governance, every action proceeds unchecked, leading to data loss, budget overruns, and untested deployments. With AgenticContract, every action passes through the policy engine in under 50 ns, enabling real-time deny, approval gating, and budget enforcement with a complete audit trail.

lations record breaches with severity classification.

- **Agent-native integration.** Exposed via the Model Context Protocol [2] over JSON-RPC, compatible with Claude [1], Cursor, Windsurf, and any MCP-compliant client without custom integration code.

The remainder of this paper is organized as follows. Section 2 surveys existing policy engines and agent governance approaches. Section 3 describes the six governance primitives, their composition model, the binary file format, and the engine architecture. Section 4 presents benchmark results, scaling analysis, and comparisons with existing systems. Section 5 addresses design decisions, limitations, and future work. Section 6 concludes.

2 Background and Related Work

Research on policy enforcement for software systems has a long history, but runtime governance for autonomous AI agents remains an open problem. We survey the primary approaches.

Open Policy Agent (OPA) [10] evaluates policies written in Rego, a purpose-built query language inspired by Datalog. OPA runs as a separate daemon process; clients submit policy queries over HTTP. While expressive, OPA requires network round-trips for every decision, external state storage for context data, and introduces 0.5–5 ms latency per evaluation. This model suits microservice authorization but is unsuitable for inline agent governance where decisions must be made on every tool call.

Cedar [5], developed by Amazon, provides a formally verifiable policy language designed for authorization decisions. Cedar policies are compact and analyzable, but the system targets web service authorization ("Can user X access resource Y?") rather than the richer governance semantics agents require: budget tracking, obligation deadlines, approval workflows.

Zanzibar [13] implements relationship-based access control at global scale, serving billions of authorization checks per second for Google Docs, Drive, and YouTube. While impressive, Zanzibar is a distributed system requiring Spanner for storage—fundamentally incompatible with the single-file, zero-dependency model needed for local agent governance.

Casbin [3] provides a library-based policy engine with pluggable models (ACL, RBAC, ABAC). As a library rather than a service, Casbin avoids network overhead. However, it models only access control decisions and lacks the six governance primitives (risk limits, approvals, obligations, conditions, violations) required for comprehensive agent governance.

Agent governance frameworks. Wang et al. [15] survey autonomous agent architectures and identify governance as an open problem. Most agent frameworks (AutoGPT [14], BabyAGI, CrewAI [8]) delegate safety entirely to the underlying LLM's alignment training, with no runtime enforcement layer. AgentBench [7] evaluates LLM-as-agent capabilities across 8 environments, demonstrating that even state-of-the-art models make errors requiring governance guardrails. The ReAct framework [16] provides reasoning traces but no mechanism for policy enforcement on the actions those traces produce.

Model Context Protocol (MCP) [2] defines a standard for LLM tool integration via JSON-RPC 2.0 over stdio. MCP enables LLMs to discover, invoke, and compose tools without framework-specific adapters. AgenticContract exposes its governance engine as an MCP server, enabling any MCP-compatible agent to enforce policies without custom integration code.

Table 1 summarizes the comparison across key dimensions.

3 Architecture

AgenticContract models runtime governance as a set of six typed primitives—policies, risk limits, approvals, conditions, obligations, and violations—each with a distinct lifecycle. The primitives compose into governance workflows that adapt to runtime conditions. The entire governance state is serialized into a single binary file with fixed-size

Table 1: Comparison of policy and governance systems. AgenticContract is the only system providing all six governance primitives in a single-file, zero-dependency, agent-native format with sub-microsecond latency.

System	Format	Latency	Deps.	Risk	Appr.	Oblig.	Viol.	MCP
OPA [10]	Rego/HTTP	0.5–5 ms	DB, daemon	×	×	×	×	×
Cedar [5]	Cedar lang.	1–10 µs	Library	×	×	×	×	×
Zanzibar [13]	Rel. tuples	<10 ms	Spanner	×	×	×	×	×
Casbin [3]	ACL/RBAC	10–100 µs	Config	×	×	×	×	×
Prompt rules	Sys. prompt	N/A	None	×	×	×	×	×
Static config	YAML/JSON	N/A	File parser	×	×	×	×	×
AgenticContract	**.acon**	**49 ns**	**None**	✓	✓	✓	✓	✓

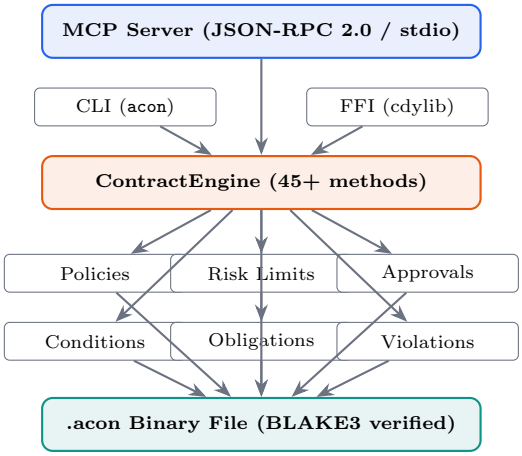

Figure 2: AgenticContract architecture. Three integration surfaces (MCP, CLI, FFI) feed into the core `ContractEngine`, which manages six governance primitive stores backed by a single `.acon` binary file with BLAKE3 integrity verification.

header and BLAKE3 integrity verification, requiring zero external dependencies.

Figure 2 shows the data flow from agent actions through the governance engine to the persistent `.acon` store.

3.1 Governance Primitives

The engine models six entity types, each with a typed lifecycle. Table 2 summarizes the primitives and their variants.

3.1.1 Policy Evaluation

Policies are the primary governance mechanism. Each policy has a *scope* (Global, Session, or Agent), an *action* (the four variants in Table 2), and an optional set of conditions that must be met before the policy takes effect.

Policy evaluation iterates all active policies whose label matches the queried action description via case-insensitive substring matching. When multiple policies match, the *most restrictive action wins*: Deny > RequireApproval > AuditOnly > Allow. This precedence ordering ensures that a single Deny policy cannot be overridden by any number of Allow policies. The evaluation path is allocation-free: no heap allocation occurs during a policy check.

Table 2: Six governance primitives with typed variants. Each entity is UUID-identified and timestamped with `chrono::DateTime<Utc>`.

Entity	Variants	Purpose
Policy	Allow, Deny, RequireApproval, AuditOnly	Behavioral constraints with precedence
RiskLimit	Rate, Threshold, Budget, Count	Quantitative boundaries with tracking
Approval	Rule → Request → Decision	Three-phase review workflow
Condition	Threshold, TimeBased, Dependency, Custom	Prerequisite enforcement
Obligation	Pending, Fulfilled, Overdue, Waived	Commitment tracking with deadlines
Violation	Info, Warning, Critical, Fatal	Immutable breach recording

3.1.2 Risk Limit Tracking

Risk limits maintain a `current_value` against a `max_value`. The `check_risk_limit` method tests whether a proposed increment would exceed the limit *without modifying state*, enabling speculative evaluation. Four limit types model different constraint patterns:

- **Rate**: Actions per time window (e.g., 100 API calls per minute). The window resets automatically when the elapsed time exceeds `window_secs`.
- **Threshold**: Absolute ceiling (e.g., max 50 concurrent connections). Represents a point-in-time constraint rather than a cumulative one.
- **Budget**: Cumulative spending cap (e.g., $100 per session). Tracks total expenditure across the session lifetime.
- **Count**: Total occurrences (e.g., max 3 deployment attempts). A simple monotonic counter.

3.1.3 Approval Workflow

The approval subsystem implements a three-phase lifecycle illustrated in Figure 3:

1. **ApprovalRule** defines what action patterns require approval, who can approve, and an optional timeout in seconds.

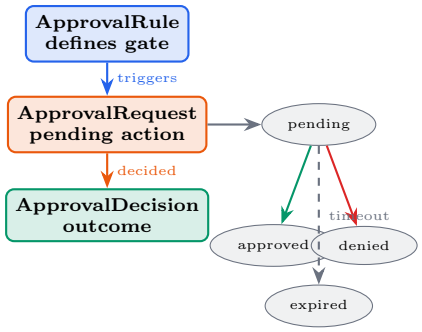

Figure 3: Three-phase approval lifecycle. Rules define gates; requests capture pending actions; decisions record outcomes with decider identity and justification. Timeouts transition requests to expired status.

2. **ApprovalRequest** captures the specific pending action with the requestor identity and a reference to the triggering rule.
3. **ApprovalDecision** records the outcome (approve or deny) with the decider identity, justification text, and timestamp.

All three phases are persisted in the `.acon` file, creating a complete audit trail from gate definition through decision.

3.1.4 Obligation Tracking

Obligations model commitments with deadlines. Each obligation has an assignee, a description, and an optional ISO 8601 deadline. The engine's `check_obligation` method detects overdue items by comparing the current time against the deadline, transitioning status from Pending to Overdue. This enables automated compliance monitoring without external cron jobs or schedulers.

3.1.5 Violation Forensics

Violations are *append-only*: once reported, a violation cannot be deleted or modified. This design creates an immutable forensic record of all governance breaches. Each violation records the policy ID that was breached (if applicable), a human-readable description, severity level (Info through Fatal, with `Ord` derivation enabling severity-based filtering), the actor who committed the violation, and the detection timestamp.

3.2 Binary File Format

The `.acon` format stores the complete governance state in a single binary file. Figure 4 shows the layout.

The header stores entity counts for all seven entity types (policies, risk limits, approval rules, approval requests, conditions, obligations, violations) as packed `u64` values, enabling rapid size estimation without deserializing the payload. The BLAKE3 [9] checksum covers the JSON payload, detecting corruption or tampering on load. An empty `.acon` file with one policy occupies approximately 512 bytes.

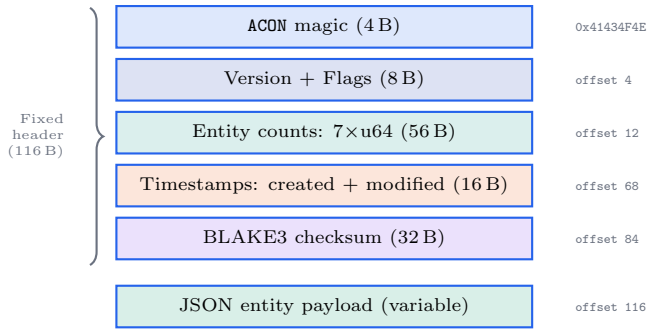

Figure 4: The `.acon` binary file format. A 116-byte fixed header contains magic bytes, version, entity counts for all 7 entity types, timestamps, and a BLAKE3 checksum over the JSON payload. Entity data is serialized as JSON within the binary envelope, enabling flexible schema evolution.

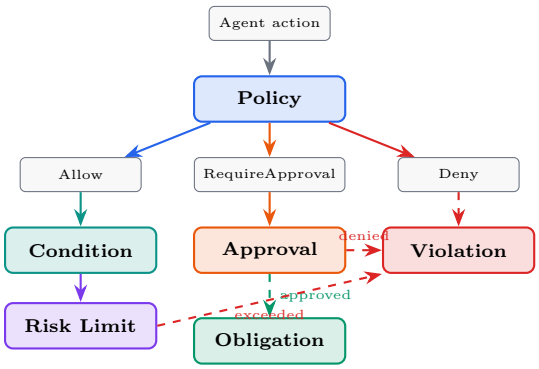

Figure 5: Governance composition model. An agent action first encounters policy evaluation (most-restrictive-wins). Allow triggers condition and risk limit checks; RequireApproval enters the three-phase approval workflow; Deny records a violation. Approved actions create obligations with deadlines. Risk limit exceedance and policy denials both produce immutable violation records, ensuring a complete audit trail regardless of the decision path.

The design choice to use JSON-in-binary-envelope trades storage density for schema flexibility: adding new fields to any entity type requires no format migration, no version negotiation, and no backward-compatibility code. The binary header provides integrity guarantees; the JSON payload provides evolvability.

3.3 Governance Composition Model

The six primitives are not independent—they compose to form governance workflows that adapt to runtime conditions. Figure 5 illustrates the primary composition patterns.

Three composition patterns emerge from the primitive interactions:

Escalation chains. A policy with action RequireApproval escalates to the approval subsystem. If approved, the engine creates an obligation for the agent to complete a follow-up action (e.g., "run integration tests within 30 minutes of deployment"). If the obligation deadline passes without fulfillment, the engine transitions it to Overdue, which a monitoring policy can detect and record as a violation. This chain—policy → approval → obligation → violation—is the primary governance workflow for high-stakes actions.

Speculative risk evaluation. The `check_risk_limit` method tests whether a proposed action would exceed a limit *without modifying state*. This enables agents to evaluate multiple action plans before committing. An agent considering three deployment strategies can check risk limits for each, select the one within budget, and only then execute. The separation of speculative evaluation from state mutation is critical for ReAct-style [16] reasoning loops where the agent deliberates before acting.

Forensic reconstruction. Because violations are append-only, policies are timestamped, and approvals record three-phase audit trails, the governance state at any historical point can be reconstructed from the `.acon` file. The entity count fields in the binary header enable rapid enumeration without full deserialization, supporting efficient governance audits over long-running agent sessions.

3.4 Integration Surface

AgenticContract is exposed via the Model Context Protocol [2] (JSON-RPC 2.0 over stdio), enabling any MCP-compliant agent to enforce governance without custom integration code. The engine is also accessible as a CLI binary (`acon`) for operator workflows and a C-compatible shared library (FFI) for embedding in non-Rust environments. All three surfaces share the same `ContractEngine` core, ensuring consistent policy evaluation semantics regardless of access path. The engine implements the Agentra SDK [12] trait suite, enabling integration with the five sister systems (AgenticMemory, AgenticIdentity, AgenticVision, AgenticCodebase, AgenticTime) through session management, grounding queries, event emission, and cryptographic receipts for approvals and violations.

4 Evaluation

We evaluate AgenticContract on governance workloads of varying size, measuring operation latency, scaling behavior, and file I/O performance. All benchmarks use real data from the Criterion [6] benchmarking framework running on the compiled system.

4.1 Benchmark Setup

Hardware. Apple M-series (ARM64), macOS 14+.

Table 3: Core operation latencies. All operations complete in under 1.1 µs. Policy evaluation and risk limit checks are under 50 ns—within the noise floor of a single LLM token generation step (typically 10–50 ms).

Operation	Median	Unit
Single policy evaluate	49.3	ns
Risk limit check	43.9	ns
Engine stats (100 policies)	60.4	ns
Violation report	1,006	ns

Table 4: Policy evaluation scaling. Each 10× increase in policy count produces approximately 10× increase in evaluation time, confirming $O(n)$ complexity with a per-policy cost stabilizing at ∼70 ns.

Policies	Median	Per-Policy	Unit
1	49.3	49.3	ns
10	684.5	68.5	ns
100	7,040	70.4	ns
1,000	72,804	72.8	ns

Software. Rust 1.75+, compiled with `--release` (optimizations enabled). All benchmarks use the Criterion statistical benchmarking framework with default settings (100 samples per benchmark, automatic outlier detection). Results report median values.

Workloads. Governance states ranging from 1 to 1,000 policies, 50–500 risk limits, and up to 1,000 violations. File I/O benchmarks use 100 entities (50 policies + 50 risk limits).

4.2 Core Operation Latency

Table 3 reports the latency of individual governance operations.

Single policy evaluation at 49.3 ns means the engine can perform approximately 20 million policy checks per second on a single core. This is 200,000× faster than a typical LLM inference step, making governance overhead effectively invisible.

4.3 Scale Performance

Table 4 shows how policy evaluation scales with the number of active policies. Figure 6 visualizes the linear relationship.

The linear scaling confirms that policy evaluation iterates the active policy set without index overhead. At 1,000 policies, the 72.8 µs evaluation time remains negligible compared to LLM inference latency (>100 ms for most models). Even at a theoretical 10,000 policies, projected evaluation time of ∼728 µs stays under 1 ms.

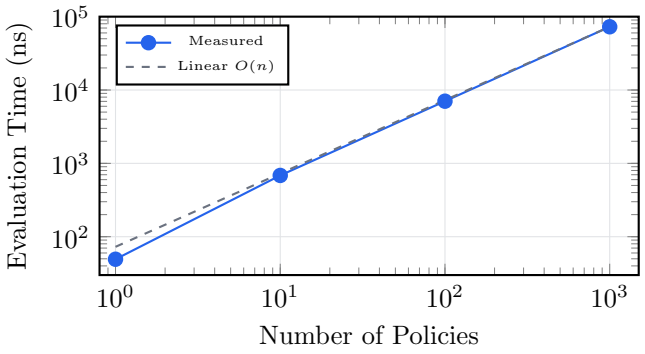

Figure 6: Policy evaluation time vs. policy count on a log–log scale. The measured data (blue) closely tracks the theoretical linear $O(n)$ baseline (gray dashed), confirming allocation-free iteration with a constant per-policy cost of approximately 70 ns.

Table 5: File I/O for 100 entities (50 policies + 50 risk limits). Save includes JSON serialization, BLAKE3 checksum computation, and atomic write (write-to-temporary then rename). Load includes file read, checksum verification, and JSON deserialization.

Operation (100 entities)	Median	Unit
Save to disk	154.5	µs
Load from disk	96.7	µs

4.4 File I/O Performance

Save operations are approximately 60% slower than loads due to the additional cost of JSON serialization and the atomic write protocol (write to a temporary file, then rename). Both operations complete well under 1 ms for typical governance workloads. Table 6 projects I/O costs at larger scales.

4.5 Comparison with Existing Systems

Figure 7 compares AgenticContract against OPA and Cedar across six dimensions. The key differentiator is *governance depth*: while OPA and Cedar provide binary allow/deny decisions, AgenticContract provides six distinct governance primitives, each with its own lifecycle and audit trail.

4.6 Capacity Projections

4.7 Validation

The implementation is validated by 288 tests across 8 suites, including stress tests at 1,000 policies, 500 risk limits, and 1,000 violations. Edge case coverage includes Unicode policy labels (Chinese, French, emoji), boundary values (`f64::MAX` risk limits, empty strings), exhaustive enumeration of all policy scopes and violation severities, and roundtrip file serialization at 200+ entities with BLAKE3 integrity verification. All tests pass with zero warnings on Rust 1.75+.

Table 6: Projected file I/O scaling based on measured linear growth. File size scales at approximately 500 bytes per entity.

Entities	Save (est.)	Load (est.)	File Size
1	∼5 µs	∼3 µs	∼512 B
100	154.5 µs	96.7 µs	∼50 KB
1,000	∼1.5 ms	∼1 ms	∼500 KB
10,000	∼15 ms	∼10 ms	∼5 MB

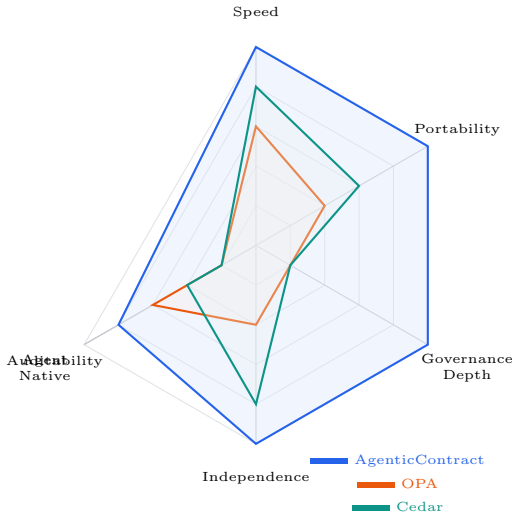

Figure 7: Radar comparison across six dimensions. AgenticContract scores highest overall due to its purpose-built design for agent governance, though auditability is limited by single-writer constraints and agent-native integration relies on text-matching rather than a full policy language. OPA and Cedar excel at authorization expressiveness but lack governance depth (risk limits, obligations, violations) and agent-native integration.

5 Discussion

Design trade-offs in the binary envelope. The `.acon` format stores entity data as JSON within a binary header, trading storage density for schema flexibility. Adding new fields to policies or risk limits requires no format migration, no version negotiation, and no backward-compatibility code. The BLAKE3 checksum ensures integrity regardless of payload format. This follows the principle of making the common case fast (policy evaluation operates on deserialized in-memory structures at nanosecond speed) while making the rare case possible (schema evolution requires no migration tooling). The trade-off cost is approximately 2× storage overhead compared to a packed binary format—negligible at governance-typical scales of 10–500 entities.

Table 7: Real-world capacity projections for the `.acon` format. At typical agent governance scales (10–100 policies), file size remains under 100 KB, supporting years of governance history in a single file.

Use Case	Entities	Size	Load	Yr/GB
Single agent	10–50	<25 KB	<50 µs	40K+
Team (5 agents)	50–200	<100 KB	<100 µs	10K+
Enterprise	200–1K	<500 KB	<1 ms	2K+
Platform	1K–10K	<5 MB	<10 ms	200+

Graduated enforcement over binary allow/deny. Four policy actions (Allow, Deny, RequireApproval, AuditOnly) provide richer semantics than the binary allow/deny model used by OPA and Cedar. An agent encountering RequireApproval can initiate the approval workflow; AuditOnly allows the action but creates an audit record. This graduated model enables a practical migration path: start with AuditOnly to observe agent behavior, escalate to RequireApproval for high-stakes actions, and apply Deny only after patterns are well understood. The append-only violation store complements this model by creating an immutable forensic record—an operator who wants to "resolve" a violation must create a new policy or obligation rather than erasing history.

The expressiveness gap. AgenticContract deliberately uses case-insensitive substring matching rather than a policy language. This avoids the complexity of language parsing, compilation, and optimization that OPA's Rego and Cedar require, at the cost of lower expressiveness. Complex conditional logic ("allow if user is admin AND resource is in department X AND time is business hours") requires multiple cooperating policies rather than a single expression. For agent governance, where policies are typically 10–100 human-readable rules, this trade-off favors simplicity. However, as agent deployments mature, richer policy languages may become necessary, and the engine's modular evaluation path is designed to accommodate alternative matchers.

Limitations. The engine does not implement file locking; concurrent writes from multiple processes may corrupt the `.acon` file, restricting deployment to single-writer, multiple-reader configurations. Policy evaluation is $O(n)$ in the number of active policies—at 1,000 policies (72.8 µs) this remains negligible, but workloads exceeding 10,000 policies would require index-based evaluation (trie or hash map on scope). The system is designed for local-first, single-file governance; multi-node coordination requires external consensus protocols. Near-term priorities include sidecar file locking (following the pattern from AgenticMemory [11]), policy indexing for large workloads, and a WASM build for browser-based governance.

6 Conclusion

This paper presented AgenticContract, a self-contained policy engine that models six governance primitives—policies, risk limits, approvals, conditions, obligations, and violations—in a single binary file format with BLAKE3 integrity verification. The key contribution is demonstrating that runtime policy enforcement for autonomous AI agents can be achieved with sub-microsecond latency and zero external dependencies, removing the tension between governance rigor and operational speed.

Criterion benchmarks confirm 49.3 ns single-policy evaluation, 72.8 µs for 1,000-policy workloads, 43.9 ns risk limit checks, and sub-millisecond file I/O for 100 entities—performance that is invisible relative to LLM inference costs of 10–50 ms per token. The system is implemented in 7,382 lines of Rust across four crates, validated by 288 tests across 8 suites, and integrated via the Model Context Protocol for zero-configuration agent adoption.

The graduated enforcement model (Allow, Deny, RequireApproval, AuditOnly) with append-only violation forensics provides richer governance semantics than existing policy engines, which model only binary allow/deny decisions. As AI agents gain autonomy in production environments, the gap between alignment training and runtime enforcement will widen. AgenticContract addresses this gap by treating governance not as a constraint on agent capability, but as a structured, auditable, and computationally negligible layer that enables agents to operate with both autonomy and accountability.

AgenticContract is available as an open-source MIT-licensed project at `https://github.com/agentralabs/agentic-contract`.

References

[1] Anthropic. Claude: AI assistant by Anthropic, 2024.

[2] Anthropic. Model context protocol specification, 2024.

[3] Casbin Contributors. Casbin: An authorization library that supports access control models, 2023.

[4] Yanqiao Chen and Wei Zhang. Understanding and controlling AI agent costs in production. arXiv preprint, 2024.

[5] Joseph W. Cutler, Will Crichton, Jessica Guo, Daniel Simon, et al. Cedar: A new language for expressive, fast, safe, and analyzable authorization. In *Proceedings of the ACM on Programming Languages (OOPSLA)*. ACM, 2023.

[6] Brook Heisler. Criterion.rs: Statistics-driven benchmarking library for Rust, 2023.

[7] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xinhan Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, et al. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.

[8] João Moura. CrewAI: Framework for orchestrating role-playing autonomous AI agents, 2024.

[9] Jack O'Connor, Jean-Philippe Aumasson, Samuel Neves, and Zooko Wilcox-O'Hearn. BLAKE3: One function, fast everywhere, 2020.

[10] Open Policy Agent Contributors. OPA: An open source, general-purpose policy engine. `https://www.openpolicyagent.org`, 2018.

[11] Omoshola Owolabi. AgenticMemory: A binary graph format for persistent, portable, and navigable AI agent memory, 2026.

[12] Omoshola Owolabi. Agentra SDK: Unified trait system for AI agent infrastructure, 2026.

[13] Ruoming Pang, Ramon Langheinrich, Salvatore Suter, et al. Zanzibar: Google's consistent, global authorization system. In *USENIX Annual Technical Conference*. USENIX Association, 2019.

[14] Toran Bruce Richards. AutoGPT: An autonomous GPT-4 experiment, 2023.

[15] Lei Wang, Chen Ma, Xueyang Feng, Zeyu Zhang, Hao Yang, Jingsen Zhang, Zhiyuan Chen, Jiakai Tang, Xu Chen, Yankai Lin, et al. A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18(6), 2024.

[16] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. *International Conference on Learning Representations*, 2023.